

Java Object Class — Complete Explanation from Java 1 to Java 26

In Java, **java.lang.Object** is the **root class of the entire class hierarchy**. Every Java class directly or indirectly extends Object, unless it is a primitive type. Arrays are also objects and inherit the methods of Object. The Java Language Specification describes Object as a superclass of all other classes. [Java Downloads](#)

```
class Employee {  
}
```

is conceptually equivalent to:

```
class Employee extends Object {  
}
```

Therefore:

```
Employee e = new Employee();
```

```
e.toString();  
e.equals(...);  
e.hashCode();  
e.getClass();
```

are possible even though these methods are not declared in Employee.

1. What is the Object class?

Definition

java.lang.Object is the **root superclass of all Java reference types**.

Its fully qualified name is:

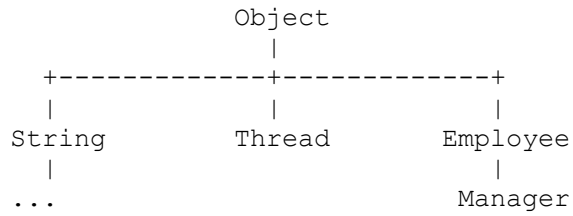
```
java.lang.Object
```

Because java.lang is automatically imported, you normally do not need:

```
import java.lang.Object;
```

The class has existed since **Java 1.0**. The current Java 26 API still identifies it as the root of the class hierarchy and marks it @since 1.0. [GitHub](#)

Basic hierarchy



For example:

```
class Employee {  
}
```

The actual inheritance relationship is:

```
Employee  
|  
Object
```

2. Why does Java need Object?

Object provides a common set of operations that every object can potentially perform.

The most important operations are:

Category	Object method
Runtime type information	<code>getClass()</code>
Equality	<code>equals()</code>
Hashing	<code>hashCode()</code>
Text representation	<code>toString()</code>
Object copying	<code>clone()</code>
Thread synchronization	<code>wait()</code>
Thread notification	<code>notify()</code>
Thread notification	<code>notifyAll()</code>
Legacy garbage-collection cleanup	<code>finalize()</code>

Java 26's API lists these nine methods as methods declared by Object. [Oracle Documentation](#)

There is also one constructor:

```
public Object()
```

and **there are no declared fields/variables in Object** in the Java 26 API.

That distinction is important:

Object has methods, but it does not provide a normal public/protected instance variable such as id, name, value, etc.

3. Object class declaration

Conceptually:

```
public class Object {
    public Object() {
    }

    public final native Class<?> getClass();

    public native int hashCode();

    public boolean equals(Object obj) {
        return this == obj;
    }

    protected native Object clone()
        throws CloneNotSupportedException;

    public String toString() {
        return getClass().getName() + "@" +
            Integer.toHexString(hashCode());
    }

    public final native void notify();

    public final native void notifyAll();

    public final void wait()
        throws InterruptedException {
        wait(0);
    }

    public final native void wait(long timeoutMillis)
```

```

        throws InterruptedException;

    public final void wait(long timeoutMillis, int nanos)
        throws InterruptedException {
        // ...
    }

    @Deprecated(...)
    protected void finalize()
        throws Throwable {
    }
}

```

This is a **conceptual representation**, not code you should copy into your own program. The actual JDK implementation contains JVM/native/intrinsic behavior. The OpenJDK source shows, for example, that `Object` is a `@since 1.0` class and that `getClass()` is a native intrinsic method. [GitHub](#)

4. Object class variables / fields

Does Object have variables?

No declared fields.

The Java 26 API does not list any fields declared by `java.lang.Object`.

So this is incorrect:

```

Object.name
Object.id
Object.value

```

There are no such fields.

Instead, `Object` supplies **methods** that operate on an object.

For example:

```
Employee employee = new Employee();
```

```

employee.getClass();
employee.hashCode();
employee.equals(otherEmployee);
employee.toString();

```

Important distinction

Concept	Object
Class	Yes
Constructor	Yes
Declared fields	No
Instance methods	Yes
Static methods	No
Final methods	Several
Native methods	Several
Deprecated method	finalize()

So when you ask for "**Object class variables and methods**", the technically accurate answer is:

Object has no declared fields, but it has one constructor and nine declared methods in Java 26.

5. Object constructor

Object()

```
public Object()
```

Definition

Creates a new Object.

Example:

```
Object obj = new Object();
```

Internally, every subclass constructor ultimately invokes the superclass constructor.

Consider:

```
class Employee {  
    Employee() {  
        System.out.println("Employee constructor");  
    }  
}
```

```
}  
}
```

Conceptually:

```
class Employee extends Object {  
  
    Employee() {  
        super();  
        System.out.println("Employee constructor");  
    }  
}
```

The compiler inserts the call to `super()` when appropriate.

6. getClass()

Syntax

```
public final native Class<?> getClass()
```

Definition

Returns the Class object representing the **runtime class** of the object.

Java 26 specifies that the returned Class represents the runtime class of the object. [GitHub](#)

Example:

```
class Employee {  
  
}  
  
public class Test {  
    public static void main(String[] args) {  
  
        Employee e = new Employee();  
  
        System.out.println(e.getClass());  
    }  
}
```

Output:

```
class Employee
```

You can also obtain its name:

```
System.out.println(e.getClass().getName());
```

Output:

Employee

For a fully qualified class:

com.example.Employee

might produce:

com.example.Employee

getClass() and polymorphism

This is particularly important:

```
class Animal {  
}
```

```
class Dog extends Animal {  
}
```

```
Animal a = new Dog();
```

```
System.out.println(a.getClass());
```

Output:

class Dog

Why?

Because:

Reference type = Animal
Actual object = Dog

getClass() reports the **actual runtime class**, not merely the declared reference type.

Why is getClass() final?

You cannot override it.

This is illegal:

```
class Employee {  
    public Class getClass() {  
        return Employee.class;  
    }  
}
```

because `Object.getClass()` is final.

7. `equals(Object obj)`

Syntax

```
public boolean equals(Object obj)
```

Definition

Determines whether two objects should be considered equal.

The default implementation in `Object` effectively performs **reference identity comparison**:

```
this == obj
```

Example:

```
Employee e1 = new Employee();  
Employee e2 = new Employee();  
  
System.out.println(e1.equals(e2));
```

Output:

```
false
```

because:

```
e1 != e2
```

They are two different objects.

8. `==` versus `equals()`

This is one of the most important Object-class concepts.

==	equals()
Operator	Method
Compares references for objects	Can compare logical state
Cannot be overridden	Can be overridden
Defined by Java language	Defined by Object
Default object behavior is identity comparison	Default implementation is identity comparison

Example:

```
String s1 = new String("Java");  
String s2 = new String("Java");  
  
System.out.println(s1 == s2);  
System.out.println(s1.equals(s2));
```

Output:

false
true

Why?

s1 == s2

asks:

Are these the same object?

while:

s1.equals(s2)

asks:

Do these String objects represent the same value?

String overrides equals().

9. Overriding equals()

Suppose:

```
class Employee {  
    int id;  
  
    Employee(int id) {  
        this.id = id;  
    }  
}
```

Then:

```
Employee e1 = new Employee(101);  
Employee e2 = new Employee(101);
```

```
System.out.println(e1.equals(e2));
```

The default Object.equals() returns:

false

because they are different objects.

We can define logical equality:

```
@Override  
public boolean equals(Object obj) {  
  
    if (this == obj)  
        return true;  
  
    if (!(obj instanceof Employee))  
        return false;  
  
    Employee other = (Employee) obj;  
  
    return this.id == other.id;  
}
```

Now:

```
System.out.println(e1.equals(e2));
```

returns:

true

10. hashCode()

Syntax

```
public native int hashCode()
```

Definition

Returns an integer hash-code value for the object.

Hash codes are particularly important in hash-based collections such as:

HashMap
HashSet
Hashtable

Basic example

```
Employee e = new Employee();  
  
System.out.println(e.hashCode());
```

You may get something such as:

746292446

The exact number should not be relied upon as a stable application identifier.

11. equals() and hashCode() contract

This is extremely important.

If:

```
a.equals(b) == true
```

then:

```
a.hashCode() == b.hashCode()
```

must also be true.

But the reverse is **not necessarily true**.

Same hash code
↓
does NOT guarantee
↓
objects are equal

Example:

a.equals(b) = true
↓
a.hashCode() == b.hashCode()

But:

a.hashCode() == b.hashCode()

does not necessarily imply:

a.equals(b) == true

This is because different objects can have the same hash code—a **hash collision**.

12. Why override hashCode() when overriding equals()?

Consider:

```
class Employee {  
    int id;  
  
    Employee(int id) {  
        this.id = id;  
    }  
  
    @Override  
    public boolean equals(Object obj) {  
        if (!(obj instanceof Employee))  
            return false;  
  
        Employee other = (Employee) obj;  
  
        return id == other.id;  
    }  
}
```

Now:

```
Employee e1 = new Employee(101);  
Employee e2 = new Employee(101);
```

```
System.out.println(e1.equals(e2));
```

returns:

true

But if hashCode() is not consistent with the equality definition, hash-based collections can behave incorrectly.

Therefore:

```
@Override  
public int hashCode() {  
    return Integer.hashCode(id);  
}
```

should accompany the equality definition.

13. toString()

Syntax

```
public String toString()
```

Definition

Returns a string representation of the object.

The default representation is generally based on:

class-name + "@" + hexadecimal hash-code

For example:

Employee@5acf9800

The Java API's contract describes toString() as returning a string representation of the object.
[Oracle Documentation](#)

Example

```
class Employee {  
}  
  
public class Test {  
  
    public static void main(String[] args) {  
  
        Employee e = new Employee();  
  
        System.out.println(e);  
    }  
}
```

This implicitly invokes:

```
System.out.println(e.toString());
```

You may see:

```
Employee@2c7b84de
```

14. Overriding toString()

Instead of:

```
Employee@2c7b84de
```

we can produce useful information.

```
class Employee {  
  
    int id;  
    String name;  
  
    Employee(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    @Override  
    public String toString() {  
        return "Employee{id=" + id +  
            ", name=" + name + "}";  
    }  
}
```

Now:

```
Employee e =  
    new Employee(101, "Anurag");
```

```
System.out.println(e);
```

Output:

```
Employee{id=101, name='Anurag'}
```

This is particularly useful for:

- debugging
 - logging
 - troubleshooting
 - displaying object state
 - unit testing
-

15. clone()

Syntax

```
protected Object clone()  
    throws CloneNotSupportedException
```

Definition

Creates and returns a copy of the object.

Java's API describes clone() as creating and returning a copy of the object. [Oracle Documentation](#)

Example:

```
class Employee implements Cloneable {  
  
    int id;  
  
    Employee(int id) {  
        this.id = id;  
    }  
  
    @Override  
    public Object clone()  
        throws CloneNotSupportedException {  
  
        return super.clone();  
    }  
}
```

Usage:

```
Employee e1 = new Employee(101);  
  
Employee e2 = (Employee) e1.clone();  
  
System.out.println(e1.id);  
System.out.println(e2.id);
```

Both contain:

101

16. Why Cloneable is needed

Cloneable is a marker interface.

```
class Employee implements Cloneable {  
}
```

If a class does not implement Cloneable, calling:

```
super.clone()
```

normally results in:

CloneNotSupportedException

The important point is that Cloneable does not itself declare clone().

Instead:

```
Object  
|  
+-- clone()
```

and:

```
Cloneable  
|  
+-- marks the class as cloneable
```

17. Shallow copy versus deep copy

Object.clone() performs a **shallow field-by-field copy**.

Suppose:

```
class Employee implements Cloneable {  
  
    int id;  
    Address address;  
  
    @Override  
    public Employee clone()  
        throws CloneNotSupportedException {  
  
        return (Employee) super.clone();  
    }  
}
```

After:

```
Employee e2 = e1.clone();
```

primitive fields are copied by value, but references are copied as references.

Conceptually:

```
e1  
|  
+-- id = 101  
|  
+-- address ----+  
                |  
                v  
            Address  
  
e2  
|  
+-- id = 101  
|  
+-- address ----+  
                |  
                v  
            same Address
```

Therefore, cloning requires careful design when mutable referenced objects are involved.

18. wait()

There are three wait signatures:

```
public final void wait()  
    throws InterruptedException
```

```
public final native void wait(long timeoutMillis)
    throws InterruptedException
public final void wait(long timeoutMillis, int nanos)
    throws InterruptedException
```

Java 26 documents all three forms. [Oracle Documentation](#)

Purpose of wait()

wait() is used for **thread coordination**.

A thread calling:

```
obj.wait();
```

must own that object's monitor.

The thread:

1. enters synchronized context,
2. calls wait(),
3. releases the object's monitor,
4. enters the waiting state,
5. waits for notification/interruption/timeout as applicable.

Example:

```
synchronized (lock) {
    lock.wait();
}
```

19. notify()

Syntax

```
public final native void notify()
```

Definition

Wakes up **one** thread waiting on the object's monitor.

Example:

```
synchronized (lock) {
```

```
    lock.notify();  
}
```

The calling thread must own the monitor.

20. notifyAll()

Syntax

```
public final native void notifyAll()
```

Definition

Wakes up **all threads** waiting on that object's monitor.

Example:

```
synchronized (lock) {  
    lock.notifyAll();  
}
```

21. wait(), notify(), notifyAll() relationship

Think of:

```
Object  
|  
+-- wait()  
|  
+-- notify()  
|  
+-- notifyAll()
```

They provide basic monitor-based thread coordination.

Example:

```
class Shared {  
    synchronized void waitForSignal()  
        throws InterruptedException {  
        wait();  
    }  
}
```

```
}  
  
synchronized void sendSignal() {  
    notify();  
}  
}
```

A modern application will often use higher-level concurrency utilities rather than directly building complex coordination around these primitives.

22. Why are `wait()`, `notify()` and `notifyAll()` final?

They are declared final, so subclasses cannot override them.

For example, this is illegal:

```
class Employee {  
    public void notify() {  
    }  
}
```

because `Object.notify()` is final.

The monitor behavior is fundamentally tied to the JVM/object monitor rather than being ordinary application-level polymorphic behavior.

23. `finalize()`

Historical syntax

```
protected void finalize()  
    throws Throwable
```

Historical purpose

`finalize()` was designed as a cleanup mechanism that could be invoked before an unreachable object was reclaimed.

Historically:

```
class Resource {
```

```

@Override
protected void finalize()
    throws Throwable {

    System.out.println("Cleanup");
}
}

```

However, **do not use this approach in new Java code.**

24. The evolution of finalize()

This is the biggest historical change to Object.

Java version	finalize() status
Java 1.0–8	Normal legacy API
Java 9	Deprecated
Java 10–17	Deprecated
Java 18	Deprecated for removal
Java 19–26	Deprecated for removal
Future Java	Intended for removal

Oracle's JDK 18 release notes explicitly identify JEP 421 as "**Deprecated Finalization for Removal**". [Oracle](#)

The Java API records @Deprecated(since="9", forRemoval=true) for Object.finalize(). [Oracle Documentation](#)

Recommended alternatives

Use:

AutoCloseable

with:

try-with-resources

or, where appropriate:

Cleaner
PhantomReference

Oracle specifically recommends alternatives such as Cleaner, PhantomReference, and explicit close()/AutoCloseable patterns. [Java Downloads](#)

25. Complete Object method table

#	Method	Access	Return	Purpose
1	getClass()	public final native	Class<?>	Gets runtime class
2	hashCode()	public native	int	Gets hash code
3	equals(Object)	public	boolean	Tests equality
4	clone()	protected	Object	Creates shallow copy
5	toString()	public	String	String representation
6	notify()	public final native	void	Wakes one waiting thread
7	notifyAll()	public final native	void	Wakes all waiting threads
8	wait()	public final	void	Waits indefinitely
9	wait(long)	public final native	void	Waits up to timeout
10	wait(long,int)	public final	void	Waits with millisecond + nanosecond precision
11	finalize()	protected	void	Legacy finalization; deprecated for removal

Why are there 11 rows when Java 26 says nine methods?

Because the API groups the overloaded methods:

```
wait()
wait(long)
wait(long,int)
```

as three methods and lists the other methods separately. Counting each overload gives **11 method signatures**. The API method summary lists the overloaded wait forms together. [Oracle Documentation](#)

26. Object methods grouped by purpose

Purpose	Methods
Runtime type	getClass()
Equality	equals()
Hashing	hashCode()
Representation	toString()
Copying	clone()
Thread waiting	wait()
Thread waiting with timeout	wait(long)
Thread waiting with precise timeout	wait(long,int)
Thread notification	notify()
Thread notification	notifyAll()
Legacy cleanup	finalize()

27. Methods you can override

These methods can normally be overridden:

```
equals()  
hashCode()  
toString()  
clone()  
finalize()
```

But their accessibility and contracts matter.

For example:

```
@Override
public boolean equals(Object obj) {
    ...
}
@Override
public int hashCode() {
    ...
}
@Override
public String toString() {
    ...
}
```

28. Methods you cannot override

These are final:

```
getClass()
wait()
notify()
notifyAll()
```

Therefore:

```
getClass() → final
wait()     → final
notify()   → final
notifyAll() → final
```

They cannot be overridden by subclasses.

29. Native methods

Some Object methods are implemented with JVM-level/native support.

Examples include:

```
getClass()
hashCode()
clone()
wait(long)
notify()
notifyAll()
```

This is because their behavior interacts closely with JVM mechanisms such as:

- object identity
- runtime type information
- object monitors
- synchronization
- JVM object representation

For example, OpenJDK's current source declares `getClass()` as a native intrinsic. [GitHub](#)

30. Object and inheritance

Consider:

```
class Animal {  
}
```

```
class Dog extends Animal {  
}
```

```
class Labrador extends Dog {  
}
```

The inheritance chain is:

```
Labrador  
↓  
Dog  
↓  
Animal  
↓  
Object
```

Therefore:

```
Labrador dog = new Labrador();
```

```
dog.toString();  
dog.equals(...);  
dog.hashCode();  
dog.getClass();
```

are all available.

31. Interfaces are different

A common misconception is:

Everything in Java extends Object.

More precisely:

Every **class and array type** has Object as its ultimate superclass.

An interface does not extend Object.

For example:

```
interface Payment {  
}
```

is not:

```
Payment  
↓  
Object
```

However, objects whose classes implement interfaces still ultimately inherit from Object.

```
class CreditCardPayment implements Payment {  
}
```

has:

```
CreditCardPayment  
↓  
Object
```

and:

```
CreditCardPayment p = new CreditCardPayment();
```

```
p.toString();  
p.hashCode();  
p.getClass();
```

remain available.

The JLS explicitly describes Object as the superclass of all classes and states that class and array types inherit its methods. [Java Downloads](#)

32. Object and arrays

Arrays are objects.

For example:

```
int[] numbers = {10, 20, 30};
```

The array itself is an object.

Therefore:

```
System.out.println(numbers.getClass());  
System.out.println(numbers.hashCode());  
System.out.println(numbers.toString());
```

are legal.

You can also use:

```
numbers.equals(otherArray);
```

but remember that the inherited default equality behavior is identity-based.

For array contents, normally use:

```
Arrays.equals(a, b);
```

rather than:

```
a.equals(b);
```

33. Object and String

String extends Object:

```
Object  
↑  
String
```

String overrides important methods:

```
equals()  
hashCode()  
toString()
```

Example:

```
String a = new String("Java");  
String b = new String("Java");  
  
System.out.println(a.equals(b));
```

Output:

true

because String provides value-based equality.

34. Object and wrapper classes

Classes such as:

Integer
Long
Double
Boolean
Character

ultimately extend Object.

For example:

Object
↑
Number
↑
Integer

and:

```
Integer a = 100;  
Integer b = 100;
```

```
System.out.println(a.equals(b));
```

uses Integer's implementation of equals() rather than simply the identity implementation inherited directly from Object.

Java's java.lang package documentation identifies Object as the root and explains the role of wrapper classes in representing primitive values as objects. [Oracle Documentation](#)

35. Object and generics

This relationship becomes very important with generics.

Before generics:

```
Object value = "Java";
```

Then:

```
String s = (String) value;
```

With generics:

```
List<String> names = new ArrayList<>();
```

you don't generally need:

```
String s = (String) names.get(0);
```

because generics provide compile-time type information.

Still, at the root of the reference-type hierarchy is:

Object

36. Object and collections

Because collections generally hold reference types, Object can represent a broad range of values:

```
List<Object> list = new ArrayList<>();
```

```
list.add("Java");
```

```
list.add(100);
```

```
list.add(10.5);
```

```
list.add(new Employee(101));
```

Now:

Object

├── String

├── Integer

├── Double

└── Employee

All can be referenced through Object.

37. Object and polymorphism

This is valid:

```
Object obj = "Java";
```

The reference type is:

Object

but the actual object is:

String

Therefore:

```
System.out.println(obj.getClass());
```

produces:

```
class java.lang.String
```

Similarly:

```
Object obj = new ArrayList<>();
```

will have a runtime class corresponding to the actual list implementation.

38. Object methods and overriding hierarchy

Consider:

```
class Employee {  
    int id;  
  
    @Override  
    public String toString() {  
        return "Employee";  
    }  
}
```

The method resolution is:

```
Object.toString()
  ↑
Employee.toString()
```

When:

```
Employee e = new Employee();
```

```
System.out.println(e);
```

the overridden method is selected.

39. Object methods and super

You can explicitly invoke the parent implementation.

```
class Employee {

    @Override
    public String toString() {

        return "Employee -> " +
            super.toString();
    }
}
```

Here:

```
super.toString()
```

invokes:

```
Object.toString()
```

because Object is the immediate superclass if Employee doesn't explicitly extend another class.

40. Object and records

Modern Java records automatically provide implementations related to:

```
equals()
hashCode()
toString()
```

For example:

```
record Employee(int id, String name) {  
}
```

Then:

```
Employee e =  
    new Employee(101, "Anurag");
```

```
System.out.println(e);
```

produces a useful record representation.

Conceptually, records provide their own implementations rather than simply relying on the raw default Object behavior.

41. Object and enums

Enums ultimately participate in the class hierarchy rooted at Object.

For example:

```
enum Status {  
    ACTIVE,  
    INACTIVE  
}
```

An enum can use:

```
Status.ACTIVE.toString();  
Status.ACTIVE.hashCode();  
Status.ACTIVE.equals(Status.ACTIVE);  
Status.ACTIVE.getClass();
```

The enum also has specialized behavior through Enum.

42. Java 1 → Java 26 evolution

The most useful way to understand the history is to recognize that **Object itself has remained remarkably stable**.

Java version	Object-class significance
--------------	---------------------------

Java 1.0	Object introduced as root class
Java 1.1	Core Object API continued
Java 1.2	Core Object methods retained; Collections framework introduced elsewhere
Java 1.3	No fundamental Object API change
Java 1.4	No fundamental Object API change
Java 5	Generics/autoboxing changed how Object is commonly used, but Object API remained fundamentally stable
Java 6	No fundamental Object API change
Java 7	No fundamental Object API change
Java 8	Lambdas/streams changed Java programming style, not Object's fundamental API
Java 9	finalize() deprecated
Java 10	No fundamental Object API change
Java 11	No fundamental Object API change
Java 12	No fundamental Object API change
Java 13	No fundamental Object API change
Java 14	No fundamental Object API change
Java 15	No fundamental Object API change
Java 16	No fundamental Object API change
Java 17	No fundamental Object API change
Java 18	Finalization deprecated for removal via JEP 421
Java 19	Same core Object API; finalization remains deprecated for removal
Java 20	Same core Object API
Java 21	Same core Object API
Java 22	Same core Object API

Java 23	Same core Object API
Java 24	Same core Object API
Java 25	Same core Object API; finalize() remains deprecated for removal
Java 26	Current API still contains finalize(), but it is deprecated for removal

The important milestones are therefore:

```

Java 1
↓
Object established
↓
Java 1–8
↓
Stable core Object API
↓
Java 9
↓
finalize() deprecated
↓
Java 18
↓
finalize() deprecated for removal
↓
Java 26
↓
Object still has the same fundamental API

```

The Java 26 API continues to list clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, and the wait overloads. [Oracle Documentation](#)

43. What changed in Java 9 regarding finalize()?

Java 9 marked finalization as deprecated.

The historical API records:

```
@Deprecated(since="9", forRemoval=true)
protected void finalize()
```

and explains that finalization can cause problems involving:

- performance
- reliability

- security
- resource leaks [Oracle Documentation](#)

So:

Java 8:

`finalize()` → legacy but not deprecated

Java 9:

`finalize()` → deprecated

Java 18:

`finalize()` → deprecated for removal

44. Java 18 and JEP 421

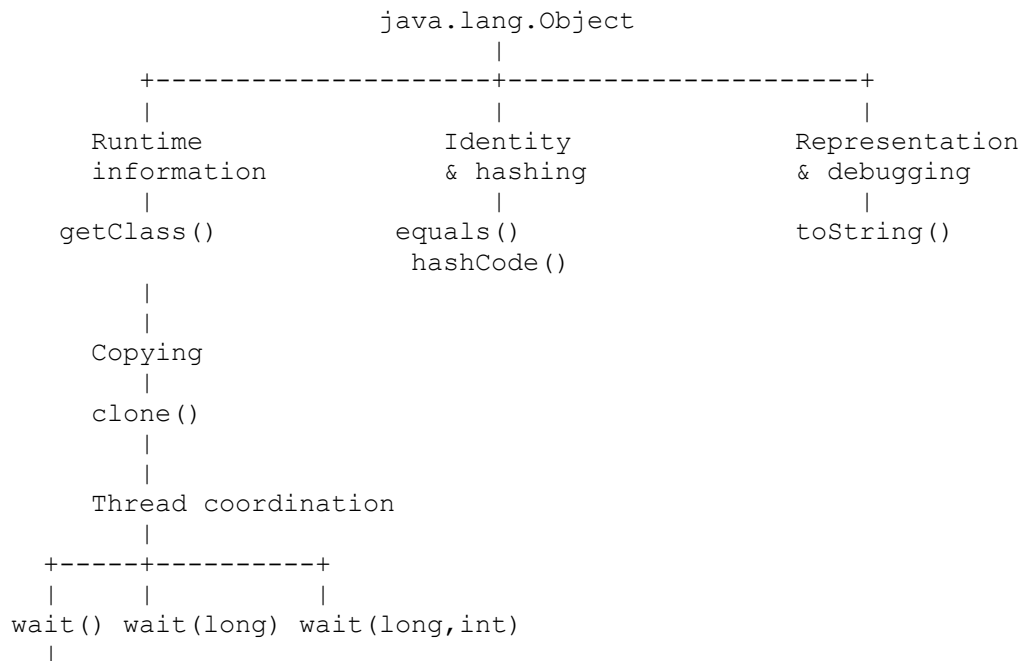
JEP 421 is especially important for understanding modern Java.

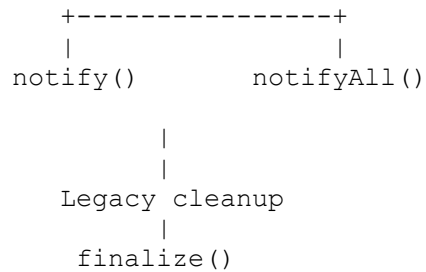
Its purpose was:

Deprecate finalization for removal.

Finalization remains in Java 26 for compatibility, but new code should not depend upon it. Oracle's JDK 18 release documentation confirms this transition. [Oracle](#)

45. Object class: complete conceptual map





46. Most important interview relationships

equals() ↔ hashCode()

equals() = logical equality

hashCode() = hash-based lookup support

If:

`a.equals(b)`

is true, then:

`a.hashCode() == b.hashCode()`

must be true.

== ↔ equals()

`==` → reference identity

`equals()` → logical equality if overridden

toString() ↔ logging

`System.out.println(object);`

essentially results in a string conversion involving:

`String.valueOf(object)`

which ultimately uses the object's string representation when the object is non-null.

Therefore, implementing a meaningful:

toString()

is extremely useful for debugging.

clone() ↔ Cloneable

Object.clone()

+

Cloneable

↓

possible cloning

But clone() produces a shallow copy unless the class explicitly implements deeper copying.

wait() ↔ notify() ↔ notifyAll()

Thread A

|

wait()

|

WAITING

|

| notify()/notifyAll()

↓

Thread A can compete to reacquire monitor

These methods are part of Java's intrinsic monitor mechanism.

47. Complete practical example

class Employee implements Cloneable {

```
    private int id;  
    private String name;
```

```
    public Employee(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }
```

```
    public int getId() {  
        return id;  
    }
```

```

public String getName() {
    return name;
}

@Override
public boolean equals(Object obj) {

    if (this == obj)
        return true;

    if (!(obj instanceof Employee))
        return false;

    Employee other = (Employee) obj;

    return id == other.id &&
        java.util.Objects.equals(name, other.name);
}

@Override
public int hashCode() {

    return java.util.Objects.hash(id, name);
}

@Override
public String toString() {

    return "Employee{" +
        "id=" + id +
        ", name=" + name + '\n' +
        '}';
}

@Override
public Employee clone()
    throws CloneNotSupportedException {

    return (Employee) super.clone();
}
}

```

Use it:

```

public class Main {

    public static void main(String[] args)
        throws Exception {

```

```

Employee e1 =
    new Employee(101, "Anurag");

Employee e2 =
    new Employee(101, "Anurag");

System.out.println("Class:");
System.out.println(e1.getClass());

System.out.println("Equals:");
System.out.println(e1.equals(e2));

System.out.println("Hash code:");
System.out.println(e1.hashCode());

System.out.println("ToString:");
System.out.println(e1);

System.out.println("Clone:");

Employee e3 = e1.clone();

System.out.println(e3);
System.out.println(e1 == e3);
System.out.println(e1.equals(e3));
}
}

```

Typical output:

Class:
class Employee

Equals:
true

Hash code:
...

ToString:
Employee{id=101, name='Anurag'}

Clone:
Employee{id=101, name='Anurag'}
false
true

Notice the particularly important distinction:

e1 == e3

is:

false

because cloning creates another object.

But:

e1.equals(e3)

is:

true

because our equals() defines equality based on the employee's state.

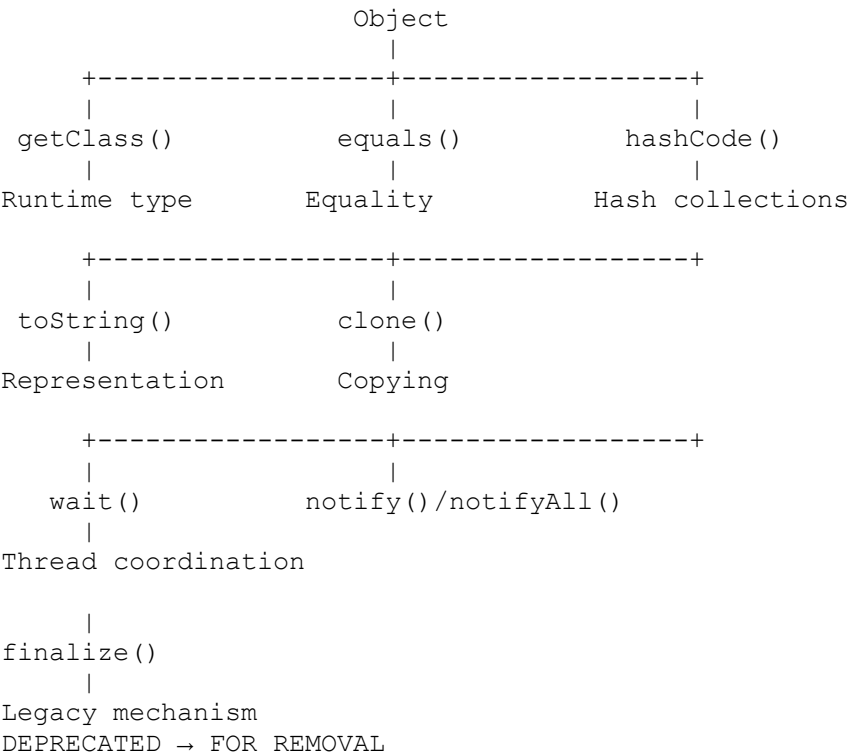
48. Final Java 26 Object-class cheat sheet

Item	Java 26 status	Override?	Main purpose
Object()	Active	N/A	Creates Object
getClass()	Active	✗	Runtime type
hashCode()	Active	✓	Hashing
equals(Object)	Active	✓	Logical equality
clone()	Active	✓	Shallow copying
toString()	Active	✓	String representation
notify()	Active	✗	Wake one waiting thread
notifyAll()	Active	✗	Wake all waiting threads
wait()	Active	✗	Wait indefinitely
wait(long)	Active	✗	Timed wait
wait(long,int)	Active	✗	Timed wait with nanos

finalize()	Deprecated for removal	Technically yes, but should not be used	Legacy finalization
-------------------	-------------------------------	---	---------------------

The Java 26 API confirms the current method set and the deprecated-for-removal status of `finalize()`. [Oracle Documentation](#)

The key thing to remember



So, from **Java 1 through Java 26**, the fundamental Object API has remained exceptionally stable. The major lifecycle change is the progressive deprecation and planned removal of **finalize()**; the other core facilities—`getClass()`, `equals()`, `hashCode()`, `toString()`, `clone()`, and monitor methods—remain fundamental parts of Java's object model. [Oracle](#)